

CYBERSPECTRE CTF 2026

Technical Challenge Write-Up

Team Winner

Member 1

Mohamed Adil

Member 2

RIASH ABDULLAH AL-DHABYANI

Scope

Solved challenges only. The report follows the actual path we used: observation, verification, exploitation or analysis, and final validation. Failed guesses that did not contribute to the final route are not presented as findings.

Prepared from the retained browser notes, terminal transcripts and evidence files from the team session.

1. Scope and Working Method

This document records the challenges that Team Winner completed during the CyberSpectre CTF. The objective of the write-up is not to list flags only. For each challenge, we show the observation that mattered, the test used to confirm it, and the exact route that led to the accepted answer. We kept the workflow practical: start with the lowest-cost hypothesis, verify it with the smallest possible test, and only expand the investigation when the evidence required it.

Exact flag values are reproduced when they were preserved in the retained transcript or later confirmed directly by the team from its submitted answers. The solve path remains based on the retained browser, terminal and evidence notes rather than reconstructed guesses.

Completed challenge matrix

Challenge	Category	Owner	Primary technique	Evidence status
Warm Up	Misc	RIASH	Base64	Exact flag retained
Reverse Uno	Misc	RIASH	String reversal	Exact flag retained
Nightglass	Misc	Mohamed	Client-side source review	Exact flag retained
Missing Page	Web	RIASH	Custom 404 inspection	Exact flag retained
Robots Don't Lie	Web	RIASH	robots.txt discovery	Team-confirmed exact flag
Not Much To See Here	Web	RIASH	JavaScript source review	Exact flag retained
Cookie Monster	Web	RIASH	Cookie inspection	Team-confirmed exact flag
front-door	Web	RIASH	Default credentials / admin portal	Exact flag retained
Auditor's Vault	Web	RIASH	HTTP response headers	Exact flag retained
Elevated	Web	Mohamed	Cookie tampering	Exact flag retained
AeroDyn Systems - Unlinked Portal	Web	Mohamed	Unlinked directory / backup leak	Exact flag retained
Needle in a Haystack	Forensics	RIASH	Targeted text search	Exact flag retained
Ghost File	Forensics	RIASH	NTFS ADS	Team-confirmed exact flag
False Appearance	Forensics	RIASH	File signature analysis	Team-confirmed exact flag
Evidence Locker	Forensics	RIASH	Cross-artifact correlation	Team-confirmed exact flag
Residual Evidence	Forensics	Mohamed	SQLite + hash + XOR + Morse	Exact flag retained
Suspect Broadcast	Crypto	RIASH	Vigenere	Exact flag retained
Binary Talk	Crypto	RIASH	Binary to ASCII	Exact flag retained
Behind the Lens	OSINT	RIASH	Image metadata	Team-confirmed exact flag
City Recon	OSINT	RIASH	Business-sign correlation	Exact flag retained
Suspect Network Log	OSINT	RIASH	Multi-log correlation + ownership	Accepted result retained
The Phantom Trail	OSINT	Mohamed	Persona / Git history OSINT	Exact flag retained
Forgotten Directory	Linux	Mohamed	Exposed Git -> SSH -> IDOR -> su	Exact flag retained
Shadowed Maintenance	Linux	Mohamed	Command injection -> cron leak -> su	Exact flag retained

Meet the Team is intentionally not included because it was still unresolved at the time this report was prepared.

2. Misc Challenges

01. Warm Up

CATEGORY

Misc | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: Base64 decoding

Starting point and reasoning

The challenge supplied one Base64-looking string. The important clue was the restricted character set and the trailing == padding. That was enough to test Base64 directly instead of trying ciphers or brute force.

Step-by-step solution

- 1. Copy the value exactly** - We copied the encoded value from the challenge instead of retyping it, because one changed character changes the decoded bytes.
- 2. Confirm the encoding** - The value used only Base64 characters and had valid padding, so a direct decode was the lowest-cost test.
- 3. Decode in the terminal** - We used the standard base64 utility and checked whether the output matched the CyberSpectre flag format.
- 4. Submit the decoded text** - The decoded output already contained the complete wrapper and was accepted without further transformation.

Key evidence / commands

```
echo "Y3liZXJzcGVjdHJlezMzc3lfcDBpbnRzX3RvX3N0NHJ0fQ==" | base64 -d
```

Validation

The terminal returned a readable CyberSpectre flag, confirming that the task was encoding only and not encryption.

Verified result: cyberspectre{33sy_p0ints_to_st4rt}

We verified the Base64 value byte-for-byte before writing this report; earlier visual guesses in the chat were discarded.

02. Reverse Uno

CATEGORY

Misc | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: String reversal

Starting point and reasoning

The supplied text looked like a flag written from right to left: braces appeared in the wrong direction and the CyberSpectre prefix was visible backwards. The structure itself was the clue.

Step-by-step solution

- 1. Look at the flag structure** - Instead of trying substitution ciphers, we first checked whether the prefix and braces became normal when the full character sequence was reversed.
- 2. Reverse the string once** - We used a simple Python slice rather than manually rearranging words, which avoids introducing spelling errors.
- 3. Check the wrapper** - After reversal, the string had a normal cyberspectre{...} structure and the inner text read logically as r34d1ng_b4ckw4rds.
- 4. Submit exactly as produced** - No case conversion or extra decoding was required.

Key evidence / commands

```
python3 - <<'PY'  
s = "<challenge-provided reversed string>"  
print(s[::-1])  
PY
```

Validation

The reversed output matched the expected flag syntax and was accepted by the platform.

Verified result: `cyberspectre{r34d1ng_b4ckw4rds}`

03. Nightglass

CATEGORY

Misc | Primary work: Mohamed Adil | Technique:
Client-side JavaScript review

Starting point and reasoning

The page presented redacted information and a credential check. Rather than treating the visual redaction as a cryptographic barrier, we treated the browser as an untrusted client and inspected the resources that implemented the check.

Step-by-step solution

- 1. Inspect loaded resources** - We checked the page source and JavaScript references instead of relying only on what was rendered on screen.
- 2. Fetch the JavaScript directly** - The guessed bundle path did not contain the values, so we requested the simple `/script.js` file referenced by the application.
- 3. Search for verification logic** - A targeted grep for flag, passphrase, secret and related terms immediately located the hard-coded expected credential and flag.
- 4. Read the controlling branch** - The script compared the user input to `Security2026!admin_access` and, on success, wrote the flag into the page. Because both values were shipped to the client, no further bypass was necessary.
- 5. Validate the exact value** - We read the script directly to avoid transcription errors and submitted the embedded flag.

Key evidence / commands

```
curl -s https://nightglass-ctf.netlify.app/script.js -o script.js  
grep -inE 'passphrase|secret|flag|Security2026|crypto|digest' script.js
```

```
const EXPECTED = "Security2026!admin_access";  
const FLAG = "cyberspectre{fake_redaction_is_not_security_2026}";
```

Validation

The controlling JavaScript itself was the authoritative source: it contained both the comparison value and the exact flag shown after successful validation.

Verified result: `cyberspectre{fake_redaction_is_not_security_2026}`

3. Web Challenges

04. Missing Page

CATEGORY

Web | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: Custom error-page inspection

Starting point and reasoning

The challenge text hinted that the missing page itself knew too much. We interpreted that literally and focused on the application's 404 handling rather than enumerating random directories.

Step-by-step solution

- 1. Trigger a controlled 404** - We requested a deliberately nonexistent path such as /404 so the custom error handler would execute.
- 2. Inspect the returned page** - The response was not a generic hosting error; it was a custom page. That difference mattered because custom error templates often contain development leftovers.
- 3. Check the visible DOM and source** - The flag appeared in the custom error content. We also opened View Source to confirm it was actually present in the response and not an extension or browser artifact.
- 4. Submit the value** - The value displayed by the custom 404 was submitted directly.

Key evidence / commands

```
https://missing-page.netlify.app/404
```

Validation

The 404 route returned the flag in the page itself and the value was accepted.

Verified result: `cyberspectre{404_custom_3rr0r}`

05. Robots Don't Lie

CATEGORY

Web | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: robots.txt path discovery

Starting point and reasoning

The title strongly suggested robots.txt. Instead of directory brute forcing, we checked the standard crawler-control file first because it can reveal paths that are intentionally hidden from search engines but still publicly reachable.

Step-by-step solution

- 1. Request robots.txt** - We opened /robots.txt on the challenge host.
- 2. Read Disallow entries** - The file disclosed a path named /secret-page.html.
- 3. Request the disclosed path** - We appended the path to the host and opened the page directly.
- 4. Read and submit the flag** - The secret page exposed the complete value `cyberspectre{r0b0ts_ar3nt_s3cr3t}`, which we copied exactly and submitted.

Key evidence / commands

```
https://robots-dont-lie.netlify.app/robots.txt
# Disclosed path:
/secret-page.html
```

```
https://robots-dont-lie.netlify.app/secret-page.html
```

Validation

The disclosed page was reachable without authentication and exposed the team-confirmed flag directly.

Verified result: cyberspectre{r0b0ts_ar3nt_s3cr3t}

06. Not Much To See Here

CATEGORY

Web | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: JavaScript source review

Starting point and reasoning

The visible page intentionally looked empty. We compared what the browser rendered with the resources referenced by the HTML instead of assuming the page content was all the application contained.

Step-by-step solution

- 1. Open the page source** - The HTML itself did not contain the flag, but it referenced script.js.
- 2. Request script.js directly** - Opening the JavaScript removed the visual layer and exposed the developer comments and variables.
- 3. Search for deployment leftovers** - A TODO comment stated that a value should be removed before production, followed immediately by a hard-coded flag constant.
- 4. Submit the hard-coded value** - Because it was included in a publicly downloadable client-side script, no authentication step was needed.

Key evidence / commands

```
https://totally-normal-site.netlify.app/script.js
```

```
// TODO: remove this before deploying to prod
const flag = "cyberspectre{v13w_s0urc3_g4ng}";
```

Validation

The flag was present verbatim in a client-side JavaScript file and was accepted.

Verified result: cyberspectre{v13w_s0urc3_g4ng}

07. Cookie Monster

CATEGORY

Web | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: Browser cookie inspection

Starting point and reasoning

The challenge name pointed to cookies, so we treated browser-side storage as the primary evidence source. This was faster and more reliable than guessing credentials or endpoints.

Step-by-step solution

- 1. Open Developer Tools** - After loading the challenge, we opened F12 and moved to Application (Storage in some browsers).

2. **Expand Cookies** - Under Storage -> Cookies we selected the challenge origin so the Name and Value columns were visible.
3. **Inspect the cookie values** - We looked for a value that matched the CyberSpectre flag format or an obviously encoded secret.
4. **Cross-check through Console** - As a second path, document.cookie can show cookies that are readable by JavaScript. This confirmed the client-side value without depending on the storage table view.
5. **Submit the recovered value** - The site cookie contained the complete value cyberspectre{c00k13s_4r3_d3l1c10us}. We submitted it directly; no additional decoding was required.

Key evidence / commands

```
document.cookie
```

Validation

The browser cookie itself contained the team-confirmed flag; no server-side exploit was required.

Verified result: cyberspectre{c00k13s_4r3_d3l1c10us}

08. front-door

CATEGORY

Web | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: Admin-route discovery and default credentials

Starting point and reasoning

The target was the Fernhollow Studio site. The first useful clue was not in the CSS or decorative JavaScript; it was the existence of an administrator entry point exposed through the site navigation/footer and the weak credential model behind it.

Step-by-step solution

1. **Map the public routes** - We reviewed links in the page and identified the administrative route (/admin), which redirected to the Administrator sign-in page.
2. **Separate presentation code from authentication** - The public site.js mostly handled interface behavior, so we stopped spending time on styling code and focused on the login workflow.
3. **Test the login request** - Using the browser Network panel, we observed the authentication request and established that the check was server-side.
4. **Check the obvious deployment weakness** - The challenge was designed around unchanged/default administrator credentials. A default credential pair was accepted by the portal.
5. **Open the authenticated admin area** - After successful login, the restricted page exposed the flag and the challenge was submitted.
6. **Preserve only verified facts** - The exact default username/password pair was not retained in the chat transcript, so this report does not invent it; the accepted flag and route are retained.

Key evidence / commands

```
/admin -> /login -> Administrator sign-in
```

Validation

The default-credential login opened the administrator area and returned an accepted flag.

Verified result: cyberspectre{d3f4ult_cr3ds_4lw4ys_w1n}

09. Auditor's Vault

CATEGORY

Web | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: HTTP response-header inspection

Starting point and reasoning

The wording “take a closer look at what the application sends back” pointed to the HTTP response, not the rendered DOM. We therefore inspected Network headers first.

Step-by-step solution

- 1. Capture the main document request** - In F12 -> Network we reloaded the page and selected the first document request.
- 2. Notice the 304 response** - The first inspection showed 304 Not Modified. That meant the browser was using a cached copy, so the full current response headers were not guaranteed to be visible.
- 3. Force a fresh request** - We enabled Disable cache and performed a hard refresh (Ctrl+F5). The document request returned 200 OK.
- 4. Read Response Headers** - In the 200 response we scanned custom x-* headers and found x-audit-trail.
- 5. Extract the value** - The custom header contained the complete flag and was submitted directly.

Key evidence / commands

```
Response Header:  
x-audit-trail: cyberspectre{h3ad3rs_t3ll_th3_full_st0ry}
```

Validation

The fresh 200 response contained the flag in x-audit-trail. Repeating the request with cache disabled reproduced it.

Verified result: `cyberspectre{h3ad3rs_t3ll_th3_full_st0ry}`

10. Elevated

CATEGORY

Web | Primary work: Mohamed Adil | Technique: Unsigned/weak role cookie tampering

Starting point and reasoning

The Corestack dashboard automatically created a guest session and its own source explained how the session was represented. The key observation was that the browser set `auth=guest` directly, while the backend function trusted the resulting role.

Step-by-step solution

- 1. Open /dashboard and inspect the inline script** - The page set `auth=guest` if no auth cookie existed, then called `/.netlify/functions/dashboard-data`.
- 2. Read the authorization branch** - The JavaScript showed that `data.role === admin` caused the master API key to be displayed.
- 3. Test whether the cookie was integrity-protected** - Instead of guessing a login password, we sent the function request with `auth=admin` as the cookie value.
- 4. Compare the server response** - The backend returned `role=admin` and the `apiKey`. This proved the role was being trusted without a cryptographic signature or server-side session lookup tied to the browser.

5. Record the returned key - The apiKey field was the challenge flag.

Key evidence / commands

```
curl -s -b 'auth=admin' https://corestack-ctf.netlify.app/.netlify/functions/dashboard-data
```

```
{"role":"admin","apiKey":"cyberspectre{cook1e_t4mp3r1ng_succ3ss}"}
```

Validation

A single modified cookie changed the server-side response from guest to admin and returned the protected value.

Verified result: cyberspectre{cook1e_t4mp3r1ng_succ3ss}

11. AeroDyn Systems - Unlinked Portal

CATEGORY

Web | Primary work: Mohamed Adil | Technique:
Unlinked content and backup disclosure

Starting point and reasoning

The public AeroDyn site did not expose the target portal through normal navigation. We treated the site as a small web application with possible unlinked deployment artifacts, then followed only paths that produced evidence.

Step-by-step solution

- 1. Review the public pages and links** - The public site led to an About/development trail rather than directly to an authentication page.
- 2. Follow the unlinked development area** - Enumeration exposed a development location (/dev_300/) and backup/snapshot content that was not linked from the normal user interface.
- 3. Inspect backup configuration** - A backup configuration file disclosed the staff-vault location and a valid staff identity: Mark Jenkins, username m_jenkins, PIN 4912.
- 4. Identify the backend endpoint** - The staff portal submitted credentials to the Netlify function /.netlify/functions/authenticate.
- 5. Call the endpoint directly** - We sent the discovered username and PIN as JSON. Direct invocation removed UI ambiguity and gave us the raw response.
- 6. Validate the returned identity and flag** - The function returned ok=true, Mark Jenkins, Senior Infrastructure Analyst, and the flag.

Key evidence / commands

```
curl -s -X POST \  
  https://aerodyn-systems.netlify.app/.netlify/functions/authenticate \  
  -H 'Content-Type: application/json' \  
  -d '{"username":"m_jenkins","pin":"4912}"'
```

Validation

The authenticate function returned a successful response for the leaked staff credentials and included the flag in JSON.

Verified result: cyberspectre{un11nk3d_d1r3ct0ry_134k_2026}

4. Forensics Challenges

12. Needle in a Haystack

CATEGORY

Forensics | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: Targeted search in a large text file

Starting point and reasoning

The evidence was a very large text file full of noise. Opening it and scrolling would be slow and error-prone, so we used the known flag prefix as a search signature.

Step-by-step solution

- 1. Identify a stable search term** - Every valid answer starts with `cyberspectre{`, giving us a precise marker with a very low false-positive rate.
- 2. Search the file programmatically** - On Windows we used Select-String; `grep` or `Ctrl+F` would produce the same result.
- 3. Inspect only the matching line** - The search returned the single line containing the flag, avoiding manual review of unrelated data.
- 4. Submit the exact match** - We copied the complete braces-delimited value from the matching line.

Key evidence / commands

```
Select-String -Path .\haystack.txt -Pattern 'cyberspectre{'
```

```
grep -i 'cyberspectre{' haystack.txt
```

Validation

The targeted search isolated a unique flag-shaped value from the noise and it was accepted.

Verified result: `cyberspectre{ctrl_f_is_powerful}`

13. Ghost File

CATEGORY

Forensics | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: Hidden attributes and NTFS Alternate Data Streams

Starting point and reasoning

After extracting the supplied RAR with WinRAR, the folder appeared empty. Because the challenge specifically required WinRAR and the environment was Windows/NTFS, we considered hidden attributes and Alternate Data Streams (ADS) before assuming the archive was blank.

Step-by-step solution

- 1. Extract with WinRAR** - We followed the challenge requirement so NTFS-specific metadata would be preserved as intended.
- 2. Show hidden filesystem entries** - In Command Prompt, `dir /a` and `attrib /s /d` revealed content that Explorer did not show in the default view, including archive and logs paths.
- 3. Enumerate alternate streams** - Inside the evidence tree we used `dir /r`. This is important because normal directory listings show the host file but not named ADS content.

4. **Locate the suspicious stream** - The listing showed recovery.txt:secret.txt:\$DATA, proving secret.txt was not a normal sibling file; it was an alternate stream attached to recovery.txt.
5. **Read the stream directly** - Windows applications can address an ADS using filename:streamname, so we opened it with Notepad.
6. **Submit the recovered flag** - The hidden alternate data stream contained cyberspectre{d0t_f1l3s_h1d3_s3cr3ts}, which was copied from recovery.txt:secret.txt and submitted.

Key evidence / commands

```
dir /a
attrib /s /d
dir /r
```

```
notepad recovery.txt:secret.txt
```

Validation

The ADS could be read directly after enumeration with dir /r, and its contents produced the team-confirmed flag.

Verified result: cyberspectre{d0t_f1l3s_h1d3_s3cr3ts}

14. False Appearance

CATEGORY

Forensics | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: Magic bytes / true file-type identification

Starting point and reasoning

The recovered artifact was named IMG_1042.jpg but would not behave like a normal JPEG. The challenge wording warned that filenames could not be trusted, so the extension was treated as a label only, not evidence of format.

Step-by-step solution

1. **Locate the actual artifact** - The first issue was path confusion in PowerShell: the file was inside a nested False_Appearance/False_appearance directory. We corrected the working directory before analysing bytes.
2. **Avoid extension-based assumptions** - We checked the content signature rather than repeatedly trying image viewers.
3. **Inspect the file header** - The intended tools were file, xxd and strings (or a PowerShell byte read). A valid JPEG should begin with the JPEG magic bytes; a different header means the .jpg suffix is false.
4. **Treat the file according to its signature** - Once the mismatch was established, the next analysis was based on the real container/content rather than the filename.
5. **Extract the embedded answer** - Following the real file content rather than the misleading .jpg extension exposed cyberspectre{d0t_f1l3s_h1d3_s3cr3ts}, which was then submitted.

Key evidence / commands

```
file IMG_1042.jpg
xxd -l 32 IMG_1042.jpg
strings IMG_1042.jpg | head
```

Validation

The key finding was the mismatch between the filename and the real content. Analysing the artifact by content exposed the team-confirmed flag.

Verified result: cyberspectre{d0t_f1l3s_h1d3_s3cr3ts}

15. Evidence Locker

CATEGORY

Forensics | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: Cross-artifact forensic correlation

Starting point and reasoning

This challenge was not solved by one hidden string. The evidence package contained notes, an employee directory, browser history, e-mail and photographs. The useful route emerged only after treating them as one timeline.

Step-by-step solution

- 1. Inventory the extracted evidence** - After extracting the archive, we located desktop_notes.txt, FORENSICS_INTAKE_NOTE.txt, browser_history.csv, company_directory.csv, final_email.eml and the evidence images. We used PowerShell recursion because the shell initially started in C:\Windows\System32.
- 2. Read the short notes before the large artifacts** - The first letters of the first four lines in desktop_notes.txt spelled READ. This told us to look for acrostics instead of treating the prose as ordinary notes.
- 3. Identify the subject** - FORENSICS_INTAKE_NOTE.txt identified Daniel Mercer, an Access Administrator in Infrastructure, missing since Tuesday.
- 4. Correlate the staff directory** - company_directory.csv linked Daniel to username dmercerc, Building B-07 and badge/employee number 4817.
- 5. Search browser history for the identity and building** - A targeted Select-String query for archive, dmercerc, flag, b-07 and 4817 revealed the Building 7 Archive web portal.
- 6. Decode the e-mail acrostic** - The leading words in final_email.eml formed MEET / AT / BUILDING7. This independently confirmed that Building 7 was intentional rather than a coincidental browser-history entry.
- 7. Use the badge credentials on the archive portal** - The site explained that internal archive authentication used employee badge credentials. dmercerc with 4817 produced access.
- 8. Follow the recovered message** - The Building 7 records included recovered_message.txt: "The meeting was never the destination. Check Room 12. — Daniel". We followed Room 12 and reused the same employee identity for the next restricted tier.
- 9. Complete the chain** - Room 12 contained the final evidence and the submitted value cyberspectre{th3_m3ss4g3_w4s_th3_m4p}. This completed the Building 7 -> Room 12 correlation chain.

Key evidence / commands

```
$target = (Get-ChildItem -Path "$HOME\Downloads" -Filter 'desktop_notes.txt' -Recurse -File
| Select-Object -First 1).DirectoryName
cd $target
Get-ChildItem -Recurse
```

```
Select-String -Path .\browser_history.csv -Pattern 'archive|dmercerc|flag|b-07|4817'
```

```
company_directory.csv -> Daniel Mercer / dmercerc / B-07 / 4817
final_email.eml acrostic -> MEET AT BUILDING 7
recovered_message.txt -> Check Room 12
```

Validation

Multiple independent artifacts converged on the same identity and location; Room 12 then exposed the team-confirmed final flag.

Verified result: cyberspectre{th3_m3ss4g3_w4s_th3_m4p}

16. Residual Evidence

CATEGORY

Forensics | Primary work: Mohamed Adil |
Technique: SQLite analysis, salted hash cracking, XOR
and Morse

Starting point and reasoning

The recovered database was the starting point, but the challenge description warned that not every value could be trusted. We therefore mapped relationships between the account row, notes and binary evidence instead of treating any single field as the answer.

Step-by-step solution

- 1. Open app.db and enumerate tables** - The relevant tables were users and notes. We focused on the compromised service account svc_backup and any note that described how its data was protected.
- 2. Extract the stored service-account hash** - The svc_backup record contained SHA-256 value 3f20fd300b3af1d625a52af4f658418ff5630b0c7ef3574e98d755726f2885ad.
- 3. Decode the hard-coded note value** - A note contained Q1lCM1ItUzNDLTc3. Base64 decoding produced CYB3R-S3C-77.
- 4. Reconstruct the password check correctly** - The stored hash matched SHA256(password + CYB3R-S3C-77), not the salt alone. Testing common candidates/wordlist entries recovered the password football.
- 5. Use the recovered password as the XOR key** - The database note indicated that the leftover blob used repeating-key XOR with the account password. XOR with football produced: "same creds unlock the rest. see attached: audio_drop.zip".
- 6. Open the attached archive** - We opened the archive in WinRAR using football, as the challenge required. It contained hint.txt and morse_signal.wav.
- 7. Interpret the audio using the hint** - The hint specified International Morse code and that the longer separators represented underscores. Decoding the groups produced D4T4B4S3_L3AK_4CC0UNT_0WN3D.
- 8. Wrap the decoded message in the platform format** - We used the lower-case CyberSpectre flag convention and submitted the result.

Key evidence / commands

```
echo Q1lCM1ItUzNDLTc3 | base64 -d
# CYB3R-S3C-77
```

```
SHA256("football" + "CYB3R-S3C-77")
= 3f20fd300b3af1d625a52af4f658418ff5630b0c7ef3574e98d755726f2885ad
```

```
XOR(blob, repeating key "football")
-> same creds unlock the rest. see attached: audio_drop.zip
```

Validation

Each stage validated the previous one: the password reproduced the stored hash, the same password correctly decrypted the blob and archive, and the Morse output formed the final flag text.

Verified result: cyberspectre{d4t4b4s3_l3ak_4cc0unt_0wn3d}

5. Crypto Challenges

17. Suspect Broadcast

CATEGORY

Crypto | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: Vigenere decryption

Starting point and reasoning

The challenge provided a ciphertext together with the key cyberspectre. Because the key was explicit and the ciphertext retained braces, digits and underscores, Vigenere was the natural first test.

Step-by-step solution

- 1. Keep non-alphabetic characters in place** - Digits, braces and underscores were structural separators, so we advanced the Vigenere key only over alphabetic characters.
- 2. Apply Vigenere decryption** - For each letter we subtracted the corresponding key letter modulo 26 while preserving case/style of the challenge text.
- 3. Read the recovered sentence** - The plaintext formed a meaningful statement about the classic Vigenere cipher and already matched the platform flag shape.
- 4. Verify with a second implementation** - We cross-checked the output with a short script/CyberChef-style Vigenere decrypt to avoid an off-by-one error in key alignment.

Key evidence / commands

```
Ciphertext: ewciikeiemii{xle3o3v3_1j_s_rp4ul1t_g1rf3s}
Key:       cyberspectre
```

Validation

The decrypted text was coherent and the exact braces-delimited value was accepted.

Verified result: cyberspectre{v1g3n3r3_1s_a_cl4ss1c_c1ph3r}

18. Binary Talk

CATEGORY

Crypto | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: 8-bit binary to ASCII

Starting point and reasoning

The challenge data consisted of binary groups. The spacing divided the stream into 8-bit bytes, which strongly suggested ASCII rather than a custom bit cipher.

Step-by-step solution

- 1. Split the stream into 8-bit groups** - A byte boundary is important; decoding a shifted bit stream would produce meaningless characters.
- 2. Convert each byte to an integer** - For example 01100011 = 99, 01111001 = 121 and 01100010 = 98.
- 3. Map integers to ASCII** - Those first three bytes spell c, y, b, immediately confirming the interpretation.
- 4. Decode the remaining bytes** - The full byte stream produced the complete CyberSpectre flag with no additional cipher layer.

Key evidence / commands

```
01100011 01111001 01100010 01100101 01110010 ...
  99      121      98      101      114
  c       y       b       e       r
```

Validation

The byte-to-ASCII conversion produced a syntactically valid flag and was accepted.

Verified result: cyberspectre{binaryls_fun}

6. OSINT Challenges

19. Behind the Lens

CATEGORY

OSINT | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: Image metadata analysis

Starting point and reasoning

The challenge supplied an image and implied that the useful information was behind what the eye could see. We therefore separated visual content from metadata and inspected the file as an evidence container.

Step-by-step solution

- 1. Preserve the original image** - We worked from the downloaded original rather than a screenshot so EXIF and application metadata would not be stripped.
- 2. Inspect metadata first** - On Windows this can be done through Properties -> Details; on a forensic workstation ExifTool gives a fuller view. We checked GPS fields, comments, author/software fields and timestamps.
- 3. Search printable embedded strings if needed** - If a useful field was not exposed by the GUI, strings can reveal comments or metadata blocks without modifying the file.
- 4. Correlate metadata with the challenge question** - The recovered metadata supplied the identifying/location clue required by the challenge and the answer was accepted.
- 5. Record the recovered metadata** - The metadata contained the final value cyberspectre{h1dden_1n_metadata}. We recorded the string from the original image and submitted it without modifying the source file.

Key evidence / commands

```
exiftool DontDigMore.jpg
strings DontDigMore.jpg | less
```

Validation

The solution came from metadata preserved in the original image, which exposed the team-confirmed flag directly.

Verified result: cyberspectre{h1dden_1n_metadata}

20. City Recon

CATEGORY

OSINT | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: Business-sign correlation and map verification

Starting point and reasoning

The image showed an industrial street with readable business signage. Instead of starting with broad reverse-image search, we used the two business names visible in the same scene because the pair is a much stronger geographic identifier than either name alone.

Step-by-step solution

- 1. Read the fixed text in the image** - The useful signs were “Auto Parts Plus” and “Baker Supply Ltd.”.
- 2. Search the two names together** - Combining both names reduced false positives from generic auto-parts businesses in other cities.
- 3. Verify the address independently** - Public business-directory information for Baker Supply Ltd. pointed to 33 Cliff Street, Nanaimo, British Columbia. The street/industrial context matched the image.
- 4. Prefer the street name over a broad city answer** - The challenge requested the location at street level, so Cliff Street was the strongest supported answer.
- 5. Submit using the expected flag normalization** - Spaces were converted to underscores and the value was lower-case.

Key evidence / commands

Observed signage:
Auto Parts Plus
Baker Supply Ltd.

Verified address:
33 Cliff Street, Nanaimo, British Columbia

Validation

The paired signage and public address independently converged on Cliff Street.

Verified result: `cyberspectre{cliff_street}`

21. Suspect Network Log

CATEGORY

OSINT | Primary work: RIASH ABDULLAH AL-DHABYANI | Technique: Multi-source log correlation and ownership attribution

Starting point and reasoning

The dataset contained firewall, DNS resolver and web-proxy logs with several deliberately suspicious-looking decoys. The reliable approach was to require one event to survive correlation across all three sources before doing any OSINT on the domain.

Step-by-step solution

- 1. Identify and reject decoy-only sessions** - Several entries used obvious DECxx session identifiers and download/update.bin paths. They looked malicious, but they did not maintain a consistent client/session/domain trail across all logs.
- 2. Build a correlation key** - We matched internal client IP, session ID, domain/IP and time window instead of relying on a single suspicious URL.

- 3. Find the consistent session** - The real chain used client 10.14.12.19 and session S9X17.
- 4. Follow the DNS event** - At 2026-08-09T04:35:38Z the resolver mapped ttgholidays.com to 185.237.145.91.
- 5. Confirm the web request** - At 04:35:45 the proxy logged GET /api/v1/sync to ttgholidays.com with the same S9X17 session, tying the DNS lookup to actual application traffic.
- 6. Pivot to ownership rather than hosting geography** - An early country guess based on infrastructure was rejected. We then checked ownership/historical WHOIS-style records for the domain instead of the current server location.
- 7. Resolve the country** - Historical attribution listed STAR COMPUTER VISION, Rajasthan, country code IN. The corrected country answer was India and the scoreboard later showed the challenge solved.

Key evidence / commands

```
Client: 10.14.12.19
Session: S9X17
DNS: ttgholidays.com -> 185.237.145.91
Proxy: GET /api/v1/sync Host: ttgholidays.com
```

Validation

The accepted answer came from ownership attribution after cross-log correlation; the earlier infrastructure-based Italy guess was discarded.

Verified result: cyberspectre{india}

The retained session notes show the corrected India submission followed by the team moving to the next challenge, and the later scoreboard marks Suspect Network Log as solved.

22. The Phantom Trail

CATEGORY

OSINT | Primary work: Mohamed Adil | Technique: Persona correlation and Git-history OSINT

Starting point and reasoning

The target was a private blog called “House & Rufus”. The login page itself did not leak credentials, and image metadata did not give the location. We pivoted from the site to the public developer persona connected to it.

Step-by-step solution

- 1. Inspect the site and client JavaScript** - app.js showed that credentials were POSTed to /.netlify/functions/login. It did not contain hard-coded credentials.
- 2. Check the favicon/photo metadata** - ExifTool mostly returned C2PA/AI-generation metadata, not a location. This eliminated a tempting but unproductive route.
- 3. Search the public persona** - The project was correlated to the handle JeradFever and related repositories including phantom-canine and cyber-phantom-99.
- 4. Review repository history, not only the current tree** - Git history exposed a deleted/backup environment file containing blog credentials. An older phantom_admin / ShadowHunter99! pair was identified as stale/decoy data; the current pair was jerad_fever / rufus_2024_secure.
- 5. Authenticate to the private blog** - Using the current credentials returned the private posts.
- 6. Extract geographic clues from the posts** - The posts referenced the yen symbol, a 10% consumption tax, a genkan, and a loop-line transport clue. These independent cultural/infrastructure details pointed to Japan rather than a single city guess.
- 7. Submit the country** - The country was normalized to lower case inside the CyberSpectre wrapper.

Key evidence / commands

```
Current credentials recovered from repository history:
BLOG_USER=jerad_fever
BLOG_PASS=rufus_2024_secure
```

Validation

The private posts contained multiple independent Japan-specific clues and the country answer was accepted.

Verified result: `cyberspectre{japan}`

7. Linux Challenges

23. Forgotten Directory

CATEGORY

Linux | Primary work: Mohamed Adil | Technique:
Exposed Git history, SSH foothold, HTTP verb tampering
and IDOR

Starting point and reasoning

The web application itself did not immediately expose privileged data, but its `.git` directory was publicly accessible. That changed the problem from web guessing to source-control recovery: Git history often preserves files that were “deleted” before deployment.

Step-by-step solution

- 1. Confirm .git exposure** - Requests to `/.git/HEAD` and the Git directory returned HTTP 200. This was enough to justify repository recovery.
- 2. Dump the exposed repository** - We used `git-dumper` to reconstruct the working repository locally instead of manually fetching objects.
- 3. Read commit history** - `git log` showed a commit named “remove leftover deploy notes”, which was exactly the kind of history worth inspecting.
- 4. Search deleted files** - `git log --all --diff-filter=D --summary` identified `deploy/credentials.txt` as deleted in commit `1f11a08`.
- 5. Recover the pre-deletion file** - `git show 1f11a08^:deploy/credentials.txt` returned `devuser / P@ssw0rd_Legacy2019`.
- 6. Use the challenge SSH service** - The recovered deployment credentials gave a shell as `devuser` on the challenge instance.
- 7. Enumerate custom local software** - Inside the host, `/opt/internal_service/app.py` was world-readable. Its own comments described two intended weaknesses and the service was bound to `127.0.0.1:9000`.
- 8. Understand the HTTP verb gate** - `GET /` deliberately returned 405, while `POST /` exposed the portal. This explained why a normal browser GET appeared closed even though the local application was reachable.
- 9. Follow the IDOR** - The source showed `/profile?id=N` without any check that the requested ID belonged to the session. User ID 8 was System Administrator.
- 10. Inspect the admin profile/source** - For ID 8, the application injected a hidden HTML comment containing `root / N0v4C0re_R00t_2024!`.
- 11. Switch users from the existing shell** - Following the challenge instruction, we used `su` rather than external SSH as root, then read `/root/root.txt`.
- 12. Record the root flag** - The root flag confirmed full compromise of the instance.

Key evidence / commands

```
git-dumper https://cyberspectre-forgotten-directory-web.chals.io/.git/ forgotten-repo
cd forgotten-repo
git log --oneline --all
git log --all --diff-filter=D --summary
```

```
git show 1f11a08^:deploy/credentials.txt
# ssh_user: devuser
# ssh_pass: P@ssw0rd_Legacy2019
```

```
Internal service findings:
GET / -> 405
POST / -> portal
/profile?id=8 -> admin profile
Hidden comment -> root / N0v4C0re_R00t_2024!
```

Validation

The chain was independently supported at every transition: exposed Git yielded valid SSH credentials; source code explained the internal weaknesses; the leaked root credential successfully switched to uid 0; root.txt returned the final flag.

Verified result: cyberspectre{v3rb_t4mp3r_1d0r_r00t}

24. Shadowed Maintenance

CATEGORY

Linux | Primary work: Mohamed Adil | Technique:
Command injection, process credential leak, local-user
pivot and password-hash cracking

Starting point and reasoning

The target exposed a host-lookup form. We first treated it as a potential command-injection sink because user input appeared to be passed to ping. The challenge also warned that SSH was not available and that discovered local credentials should be used from the existing shell, which influenced the privilege-escalation plan.

Step-by-step solution

- 1. Confirm operating-system command injection** - Submitting host=127.0.0.1; id appended a second shell command and returned uid=33(www-data). This established code execution with the Apache worker account.
- 2. Read the application source** - /var/www/html/index.php showed the vulnerable construction: \$cmd = "ping -c 2 " . \$host . " 2>&1" followed by shell_exec(\$cmd). There was no validation or shell escaping.
- 3. Look for challenge-specific local assets** - /etc/passwd revealed www-ctf and n0flow. /opt/maintenance contained maintenance.db owned root:maintainers (0640) and sync_status.sh owned root:root (0700). n0flow belonged to maintainers, so that account would be able to read the database.
- 4. Use the healthcheck clue** - healthcheck.php stated that it was used by the maintenance sync job. This connected the web application to the privileged maintenance workflow.
- 5. Read the container endpoint** - /endpoint.sh contained a decisive operational comment: cron drove a hidden root-crontab credential leak. It also showed that SSH existed only as a loopback/internal path, consistent with the challenge note.
- 6. Observe the root cron process** - A process watcher caught root executing /opt/maintenance/sync_status.sh and a curl command to http://127.0.0.1/healthcheck.php. A normal ps sample showed the -u argument blank because curl quickly sanitised credentials in argv.

- 7. Increase observation resolution** - We used a short-lived Python /proc watcher polling new root PIDs every few milliseconds. This captured the curl command before sanitisation: n0flow:Qu3ueFlow!Sync99.
- 8. Switch to n0flow without external SSH** - Because the web command context had no interactive TTY, piping the password immediately to su failed. We created a pseudo-terminal with script and delayed the password by one second; id then confirmed uid=1001(n0flow) and membership in maintainers.
- 9. Read the maintenance database** - As n0flow, sqlite3 could read maintenance.db. The credentials table contained the same n0flow password and a yescrypt-style root password hash.
- 10. Crack the root hash offline** - We saved the hash to ~/root.hash and ran John the Ripper with Kali's rockyou.txt. The hash cracked in seconds as trustno1.
- 11. Switch to root through the same pseudo-TTY method** - Using trustno1 with script -qec and su - root returned uid=0 and allowed /root/root.txt to be read.
- 12. Clean up temporary watcher files** - The /tmp/.cs_* process-watch files created for observation were removed after use, important because the instance was shared.

Key evidence / commands

```
curl -skG "$TARGET/" --data-urlencode 'host=127.0.0.1; id'
# uid=33(www-data) gid=33(www-data)
```

```
$cmd = "ping -c 2 " . $host . " 2>&1";
$result = shell_exec($cmd);
```

```
Captured before curl argv sanitisation:
curl -s -o /dev/null -u n0flow:Qu3ueFlow!Sync99 http://127.0.0.1/healthcheck.php
```

```
(sleep 1; printf "Qu3ueFlow!Sync99\n") | script -qec "su - n0flow -c \"id\"" /dev/null
```

```
credentials table:
sync_worker | n0flow | Qu3ueFlow!Sync99
system | root | $y$j9T$vKNoIFgCjXNAkUZHQ5g1r.$UK4bCkLO6w2g34qc6O6U4ROTxJwcSrUTsa.ijTj5Zy/
```

```
john --format=crypt --wordlist=/usr/share/wordlists/rockyou.txt ~/root.hash
# trustno1
```

```
(sleep 1; printf "trustno1\n") | script -qec "su - root -c \"id; cat /root/root.txt\"" /dev/null
```

Validation

The last command returned uid=0(root) and the root flag. This also validated the entire escalation chain from web injection to local-user credential disclosure to root password recovery.

Verified result: cyberspectre{n0_cr0n_c4n_h1de_f0r3v3r}

8. Closing Notes

Across the solved set, the most useful habit was to follow evidence instead of the apparent difficulty of the challenge. Several “hard” paths became short once the correct trust boundary was identified: client-side code in Nightglass, a role cookie in Elevated, Git history in Forgotten Directory, and transient process arguments in Shadowed Maintenance. The forensics and OSINT tasks rewarded the same approach in a different form: correlate independent artifacts, reject decoys that do not survive cross-checking, and preserve the original file whenever metadata matters.

We also kept failed guesses out of the final reasoning unless they changed the investigation. For example, the Suspect Network Log country was corrected only after distinguishing infrastructure geography from ownership, and the Shadowed Maintenance process watcher was redesigned after ordinary ps output showed that curl had already sanitised its credential argument. These corrections are included because they explain why the final method is reliable rather than presenting the result as a lucky guess.

Meet the Team remains outside this report because it had not been solved and verified. The document therefore reflects completed work only.